

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#define MAX_VERTICES 10
#define INFINITY 999
// Function prototypes
void dijkstra(int graph[MAX_VERTICES][MAX_VERTICES], int n, int src);
int main() {
    int graph[MAX_VERTICES][MAX_VERTICES], n, src;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    // Input the adjacency matrix
    printf("Enter the cost adjacency matrix (enter %d for no direct path):\n", INFINITY);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            scanf("%d", &graph[i][j]);
            if (graph[i][j] == 0 && i != j) {
                graph[i][j] = INFINITY; // Assigning infinity where there is no direct edge
            }
        }
    }
    printf("Enter the source vertex (starting from 0): ");
    scanf("%d", &src);
    dijkstra(graph, n, src);
    return 0;
}

void dijkstra(int graph[MAX_VERTICES][MAX_VERTICES], int n, int src) {
    int dist[MAX_VERTICES]; // Array to store the shortest distance from source to i
    int visited[MAX_VERTICES]; // Array to mark visited nodes
    int parent[MAX_VERTICES]; // Array to store the parent of each node in the shortest path tree
    // Initialize all distances as INFINITY and visited[] as false
    for (int i = 0; i < n; ++i) {
        dist[i] = INFINITY;
        visited[i] = 0;
    }
    // Distance of source vertex from itself is always 0
    dist[src] = 0;
    parent[src] = -1; // Source has no parent
    // Find shortest path for all vertices
    for (int count = 0; count < n - 1; ++count) {
        // Pick the minimum distance vertex from the set of vertices not yet processed
```

```

int u = -1;
for (int v = 0; v < n; ++v) {
    if (!visited[v] && (u == -1 || dist[v] < dist[u])) {
        u = v;
    }
}
// Mark the picked vertex as visited
visited[u] = 1;
// Update dist value of the adjacent vertices of the picked vertex
for (int v = 0; v < n; ++v) {
    if (!visited[v] && graph[u][v] && dist[u] != INFINITY && dist[u] + graph[u][v] < dist[v]) {
        dist[v] = dist[u] + graph[u][v];
        parent[v] = u;
    }
}
// Print the shortest paths
printf("\nShortest Paths from Source Vertex %d:\n", src);
for (int i = 0; i < n; ++i) {
    if (i != src) {
        printf("Path to vertex %d: Cost = %d | Path = %d", i, dist[i], i);
        int p = parent[i];
        while (p != -1) {
            printf(" <- %d", p);
            p = parent[p];
        }
        printf("\n");
    }
}
}

```